ПРОЄКТ «БУДИНОК З ПРИВИДАМИ» (HAUNTED HOUSE)

Повний практичний посібник та шпаргалка для захисту проєкту з Three.js
Пояснення архітектури, графіки та коду максимально простою людською мовою

1. Про що цей проєкт (Суть на 1 хвилину)

Цей проєкт — інтерактивна 3D-сцена нічного «будинку з привидами», згенерована повністю програмно прямо у браузері за допомогою WebGL та бібліотеки Three.js. Тут немає жодної імпортованої готової 3D-моделі з 3ds Max чи Blender! Кожен елемент (стіни, дах, двері, кущі, надгробки) створено виключно з базових математичних фігур (куби, конуси, площини, сфери) і деталізовано сучасними текстурами високої роздільності (PBR).

Головні переваги проєкту для викладача:
1. Повна динаміка: 60 кадрів/сек, літають три кольорові привиди з реальними розрахунками світла і тіней.
2. Професійні PBR-текстури: рельєфні кам'яні плити, об'ємна стара цегла, дерев'яні двері з картою висоти.
3. Оптимізація ресурсів: перехід на WebP зменшив вагу текстур з 13 МБ до 1.6 МБ (швидкість завантаження зросла у 8 разів!).

2. Стек технологій (Що використано і навіщо)

• **HTML5 Canvas:** Полотно сторінки <canvas class="webgl">, на якому WebGL малює кожен піксель 3D-графіки.

• **JavaScript (ES6 Modules):** Мова логіки проєкту. Створює об'єкти, завантажує текстури, налаштовує світло та циклічно малює кадри анімації.

• **Three.js:** Найпопулярніший 3D-фреймворк. На чистому WebGL для малювання одного кубика треба писати 300+ рядків коду матричної алгебри та шейдерів. Three.js надає зручні абстракції: Scene, Camera, Mesh, Material, Light.

• **Vite:** Швидкий бандлер і локальний веб-сервер. Чому не можна відкрити index.html подвійним кліком? Браузер з міркувань безпеки (політика CORS) забороняє завантажувати картинки й текстури напряму з диска (file://). Vite запускає локальний HTTP-сервер http://localhost:5173 і роздає ресурси безпечно.

• **lil-gui:** Випадаюче вікно налаштувань у кутку екрана. Дозволяє змінювати висоту рельєфу підлоги (displacementScale, displacementBias) на льоту без переписування коду.

• **Three.js Addons (Sky, Timer, OrbitControls):** Sky генерує процедурне небо, Timer розраховує стабільний час анімації, OrbitControls дозволяє крутити камеру мишкою.

3. Анатомія 3D-світу в Three.js (Базова структура)

Будь-яка 3D-сцена у Three.js тримається на 4 базових елементах:

• **Сцена (Scene):** Віртуальний світ або коробка, куди додаються всі об'єкти: будинок, ліхтарі, привиди (scene.add(object)).

• **Камера (PerspectiveCamera):** Очі користувача у віртуальному світі. Перспективна камера імітує людське око: далекі об'єкти здаються меншими. Параметри: кут огляду (75°), пропорції вікна, мінімальна (0.1) та максимальна (100) дистанції відсікання.

• **Рендерер (WebGLRenderer):** Художник. Він бере сцену, дивиться на неї крізь камеру і 60 разів на секунду відмальовує фінальну картинку на екрані.

• **Меш (Mesh = Geometry + Material):** Будь-який видимий предмет у просторі: Geometry — це каркас форми з трикутників, а Material — це "шкіра", колір та реакція на освітлення.

4. Як побудовано кожен об'єкт сцени

1. Земля та рельєфна підлога (Floor)

• Геометрія: PlaneGeometry(20, 20, 100, 100) — площина 20x20 метрів, розбита на 100x100 квадратів. Багато вершин потрібно для карти висоти (displacementMap), яка фізично вигинає поверхню, утворюючи справжні горбики й каміння.
• Поворот: оскільки площина створюється вертикально, її повернули на -90° (-Math.PI * 0.5) по осі X, поклавши горизонтально.
• Альфа-мапа (alphaMap): чорно-біла радіальна текстура. Центр білий (непрозорий), краї чорні (прозорі). Завдяки цьому земля не обривається різким квадратом, а плавно зникає в темряві.

2. Контейнер будинку (Group `house`)

Усі елементи споруди об'єднано в спільну групу new THREE.Group(). Це дозволяє рухати або обертати весь будинок цілком як єдиний об'єкт.

• **Стіни (walls):** BoxGeometry(4, 2.5, 4) — коробка шириною 4м, висотою 2.5м, довжиною 4м. Її піднято по осі Y на 1.25м (половина висоти), щоб дно стояло точно на нульовому рівні підлоги.

• **Дах (roof):** ConeGeometry(3.5, 1.5, 4) — конус з 4 гранями формує 4-гранну піраміду. Дах повернуто на 45° (Math.PI * 0.25) навколо осі Y, щоб його грані точно збіглися зі стінами.

• **Двері (door):** PlaneGeometry(2.2, 2.2, 100, 100) з текстурою дощок. Встановлені на position.z = 2 + 0.01. Чому +0.01? Якби координата була рівно 2, площина дверей збіглася б із гранню стіни. Відеокарта плуталась би, що малювати попереду (ефект мерехтіння Z-fighting). Зсув усього на 1 см повністю усуває цей артефакт.

• **Кущі (bushes):** 4 сфери (SphereGeometry) біля ґанку. Вони використовують ОДНУ спільну геометрію та ОДИН матеріал (важливий патерн оптимізації пам'яті).

3. Кладовище та 30 надгробних плит (Graves)

Створено 30 плит (BoxGeometry(0.6, 0.8, 0.2)), розташованих по колу навколо будинку за формулами шкільної тригонометрії:
• Кут: angle = Math.random() * Math.PI * 2 (випадковий напрямок від 0 до 360°).
• Радіус: radius = 3 + Math.random() * 4 (дистанція від 3 до 7 метрів від центру будинку).
• Координати: x = sin(angle) * radius, z = cos(angle) * radius.
• Кожному надгробку задано випадковий нахил по всіх осях: (Math.random() - 0.5) * 0.4, щоб кладовище виглядало занедбаним.

5. Текстури PBR — Що це таке простими словами

PBR (Physically Based Rendering) — це сучасний стандарт 3D-графіки, який точно повторює фізику поведінки світла в реальному світі. Замість однієї пласкої картинки матеріал використовує набір карт:

• **Color / Diffuse (колір):** Власне картинка матеріалу (цегла, дошки, листя). Для неї обов'язково встановлюється colorSpace = THREE.SRGBColorSpace, щоб кольори не виглядали бляклими.

• **ARM (AO + Roughness + Metalness):** Трюк оптимізації! В один графічний файл запаковано три різні монохромні карти: канал Red = Ambient Occlusion (тіні в тріщинах), Green = Roughness (шорсткість/матовість), Blue = Metalness (металічність). Це зменшує кількість звернень до текстур у 3 рази.

• **Normal Map (нормалі):** Фіолетово-блакитна карта рельєфу. Вона не змінює форму об'єкта у просторі, але змінює кут відбиття світла. Створює враження мільйона дрібних деталей і тріщин взагалі без навантаження на геометрію.

• **Displacement Map (карта висот):** На відміну від Normal, вона фізично пересуває вершини 3D-сітки вгору та вниз, роблячи поверхню насправді об'ємною (використано для каміння підлоги та дощок дверей).

6. Освітлення та Тіні (Світловий дизайн)

Атмосферу хоррору забезпечує дворівнева схема освітлення:

• **AmbientLight (фонове світло):** Блідо-блакитне розсіяне світло (#86cdff, інтенсивність 0.275). Підсвічує темні сторони будинку, щоб вони не ставали суцільно чорними плямами.

• **DirectionalLight (місяць):** Холодне направлене світло (#86cdff, інтенсивність 1). Світить ззаду будинку паралельними променями та формує довгі нічні тіні.

• **PointLight біля дверей (ліхтар):** Тепле затишне світло (#ff7d46, інтенсивність 5). Імітує лампочку над дверима й створює контраст із холодним довкіллям.

• **PointLight привиди (3 штуки):** Три рухомих джерела світла (фіолетовий #8800ff, рожевий #ff0088, червоний #ff0000). Літають по двору й підсвічують могили.

Робота з тінями

• Увімкнено renderer.shadowMap.enabled = true з алгоритмом м'яких тіней THREE.PCFSoftShadowMap.
• Об'єкти, що кидають тіні (castShadow): місячне світло, привиди, стіни, дах, могили.
• Об'єкти, що приймають тіні (receiveShadow): підлога, стіни, могили.
• Оптимізація: розмір карти тіней зменшено до 256x256. Це дає красиві м'які розмиті тіні хоррору та не навантажує відеокарту.

7. Атмосфера: Небо (Sky) та Туман (Fog)

• **Шейдерне Небо (Sky.js):** Спеціальний модуль Three.js, який математично розраховує розсіювання світла в атмосфері (закони Релея та Мі). Положення сонця опущено за лінію горизонту, створюючи глибокі сутінки.

• **Експоненційний туман (FogExp2):** Колір #04343f, щільність 0.1. Туман починається плавно і густішає з відстанню, маскуючи межі підлоги й створюючи ефект нескінченного простору.

8. Анімація привидів: Математика руху

Привиди літають у функції tick(), яка викликається 60 разів на секунду через requestAnimationFrame:
1. Кругова орбіта (X та Z): розраховується через x = cos(angle) * R, z = sin(angle) * R. У кожного привида свій радіус (4, 5, 6 метрів), швидкість і напрямок (ghost2 летить у зворотний бік завдяки знаку мінус).
2. Хвилеподібні коливання по висоті (Y): щоб політ здавався живим і хаотичним, перемножено три синусоїди з різними частотами:

y = sin(angle) * sin(angle * 2.34) * sin(angle * 3.45)

Завдяки різним коефіцієнтам траєкторія стає нелінійною: привид то плавно ширяє, то пірнає крізь землю.

Чому Timer замість Clock?
У Three.js клас Clock мав баги: при згортанні вкладки браузера час продовжував накопичуватись, і при поверненні об'єкти некеровано стрибали. Новий клас Timer працює стабільно і коректно зупиняє розрахунок, коли сторінка неактивна.

9. ТОП-10 запитань викладача на захисті (Готові відповіді)

Питання №1: Чому будинок складено з примітивів, а не завантажено готовою 3D-моделлю (.gltf / .obj)?

[Простими словами]: Мета лабораторної — навчитися працювати з базовими формами Three.js, ієрархією об'єктів (Group) та налаштуванням текстур власноруч.

[Для викладача (технічно)]: Створення процедурної сцени через примітиви демонструє повне розуміння геометричного конвеєра Three.js, економить пам'ять і дозволяє динамічно управляти кожним вузлом через код.

Питання №2: Що таке Z-fighting і як ви його вирішили?

[Простими словами]: Це мерехтіння двох поверхонь, які знаходяться на одній і тій самій глибині (відеокарта не знає, яка попереду).

[Для викладача (технічно)]: Вирішено для дверей і стіни: координату дверей зміщено вперед по осі Z на +0.01м (z = 2.01 замість 2.0).

Питання №3: Чому колірна текстура має colorSpace = SRGBColorSpace, а карти нормалей і ARM — ні?

[Простими словами]: Кольорові зображення мають гамма-корекцію для людського ока. А нормалі й ARM — це чисті цифрові вектори та коефіцієнти для формул шейдера.

[Для викладача (технічно)]: Текстури даних повинні інтерпретуватися строго лінійно (LinearColorSpace). Застосування sRGB до карти нормалей спотворило б вектори відбиття світла.

Питання №4: Як ви оптимізували створення кущів і могил?

[Простими словами]: Ми створили лише 1 екземпляр геометрії та 1 екземпляр матеріалу, і використали їх повторно для всіх 30 надгробків.

[Для викладача (технічно)]: Використання спільних екземплярів BufferGeometry та Material мінімізує кількість викликів малювання (draw calls) і тримає в VRAM лише один буфер вершин.

Питання №5: Чому ви перейшли на формат WebP для текстур?

[Простими словами]: WebP важить майже у 8 разів менше за JPG/PNG при тій самій якості. Загальний розмір проєкту впав з 13 МБ до 1.6 МБ.

[Для викладача (технічно)]: Формат WebP забезпечує високу компресію без втрати помітних деталей, що зменшує навантаження на мережу та час завантаження сторінки користувачем.

Питання №6: Чому розмір тіньової мапи встановлено 256x256, а не 2048x2048?

[Простими словами]: Для нічного хоррору потрібні м'які розмиті тіні, а висока роздільність зробила б їх різкими та перевантажила GPU.

[Для викладача (технічно)]: Кожне джерело світла з castShadow створює окремий прохід рендерингу (shadow pass). Розмір 256 разом із PCFSoftShadowMap оптимізує FPS без втрати атмосфери.

Питання №7: Чому для ліхтаря біля дверей тіні вимкнено?

[Простими словами]: Він знаходиться впритул до стіни і його тіні майже не помітні, проте кожне світло з тінню уповільнює роботу сцени.

[Для викладача (технічно)]: Вимкнення тіней на другорядних точкових джерелах світла зберігає ресурси для динамічних тіней від привидів.

Питання №8: Чим відрізняється Fog від FogExp2?

[Простими словами]: Звичайний Fog починається з певної дистанції (near) і стає суцільним на іншій (far). А FogExp2 густішає експоненційно з відстанню, що значно реалістичніше.

[Для викладача (технічно)]: FogExp2 розраховує видимість за формулою експоненційного згасання e^(-density * distance), що точно моделює природний туман.

Питання №9: Як працює формула траєкторії польоту привидів?

[Простими словами]: По горизонталі вони кружляють по колу завдяки sin і cos. А по вертикалі висота змінюється через перемноження 3 синусів із різною швидкістю, що робить рух непередбачуваним.

[Для викладача (технічно)]: Суперпозиція гармонік y = sin(t)*sin(2.34t)*sin(3.45t) генерує псевдовипадкову гладку криву без використання важких фізичних рушіїв.

Питання №10: Що робить alphaMap на підлозі і як працює розчинення країв?

[Простими словами]: Це чорно-біла радіальна картинка. У центрі білий колір (непрозорий), до країв чорний (прозорий). Матеріал із transparent: true робить чорні пікселі невидимими.

[Для викладача (технічно)]: alphaMap керує альфа-каналом фрагментного шейдера. У поєднанні з FogExp2 це повністю розмиває геометричні межі сцени.

---

10. Шпаргалка: Зразок промови для захисту (2-3 хвилини)

«Добрий день! До вашої уваги мій мініпроєкт із комп'ютерної графіки — інтерактивна 3D-сцена "Haunted House" (Будинок з привидами), створена за допомогою бібліотеки Three.js та WebGL.

Головне завдання проєкту — побудувати цілісний атмосферний 3D-світ без використання готових імпортованих моделей, виключно з базових математичних примітивів:

1. Архітектурна частина: будинок зібрано в контейнер Group зі стін (BoxGeometry), 4-гранного даху (ConeGeometry) та вхідних дверей із захистом від артефакту Z-fighting.

2. Оточення: процедурно розставлено 30 надгробних плит із використанням тригонометричного розподілу по колу та випадковими кутами нахилу. Геометрія та матеріали могил і кущів оптимізовані через спільні екземпляри.

3. PBR-текстури: реалізовано повний пайплайн Physically Based Rendering (Color, Normal, ARM-карти та Displacement-карту підлоги). Усі асети оптимізовано у формат WebP зі зменшенням розміру з 13 МБ до 1.6 МБ.

4. Світло та атмосфера: сцена освітлена холодним місячним світлом, теплим ліхтарем над дверима та процедурним атмосферним небом Sky із симуляцією розсіювання Релея-Мі. Межі площини приховані експоненційним туманом FogExp2.

5. Анімація: по нелінійних орбітах рухаються три привиди, траєкторія яких формується суперпозицією тригонометричних гармонік.

Проєкт опубліковано на GitHub та задеплоєно на Vercel. Дякую за увагу, готовий відповісти на запитання!»

Ключові терміни, які треба вимовити впевнено:
• PBR (Physically Based Rendering) — фізично коректний рендеринг матеріалів.
• ARM Map — об'єднання карт Ambient Occlusion, Roughness та Metalness в одну картинку.
• Z-fighting — конфлікт глибини двох суміжних поверхонь.
• Displacement Map — реальне зміщення вершин геометрії на відміну від імітації в Normal Map.
• PCFSoftShadowMap — м'які розмиті тіні.
• Timer замість Clock — виправляє накопичення часу у згорнутих вкладках.